



Relatório - BD2- Matrix



Professor: Mirella Silva Junqueira



Alunos:

Ézio Enrique Fiúza Ribeiro

Marcelo Augusto Naves Cruz

Gil Antony Borba Veloso Araújo Oliveira

Sinopse da obra:

“O jovem programador Thomas Anderson é atormentado por estranhos pesadelos em que está sempre conectado por cabos a um imenso sistema de computadores do futuro. À medida que o sonho se repete, ele começa a desconfiar da realidade. Thomas conhece os misteriosos Morpheus e Trinity e descobre que é vítima de um sistema inteligente e artificial chamado Matrix, que manipula a mente das pessoas e cria a ilusão de um mundo real enquanto usa os cérebros e corpos dos indivíduos para produzir energia.”

Este banco de dados em questão é pensado como sendo utilizado pelas máquinas dominantes do universo de Matrix. O intuito desse banco é armazenar abstrações da realidade criada, focando em apenas alguns aspectos simulados pela mundo virtual apresentado no filme. No filme, por se tratar de uma simulação completa da realidade, seria inviável criar entidades para cada objeto possível e cada interação possível entre eles, além disso, foi decidido armazenar apenas dados predeterminados. Por conta disso, apenas os aspectos a serem apresentados e seus relacionamentos foram selecionados para emular o suposto banco.

Entidades:

Humanos: A tabela Humanos é responsável por armazenar dados básicos de humanos, que são seres reais (não criados pela IA)

Entidades: Armazenam dados predeterminados sobre criaturas, não inteligentes, criadas pelas IA

IA/Agentes: Armazenam dados base sobre as Inteligências Artificiais que foram criadas pelas maquinas dominantes, com o propósito de cuidar da realidade virtual. A Especialização para Agentes foi necessária pois eles representam um grupo não abrangente e com um propósito mais específico do que as outras IAs.

Eventos: Armazenam dados sobre eventos pré-determinados; Exemplos: Desastres naturais, eclipses, etc...

Estruturas: Armazenam dados sobre objetos, geralmente grandes e imóveis, com os quais os humanos não tem tanta interação.

Local: Armazenam informações básicas sobre regiões do mundo virtual

Habilidades: Armazenam representações de habilidades que as IAs podem possuir, como por exemplo a manipulação de Estruturas, Eventos, e Entidades.

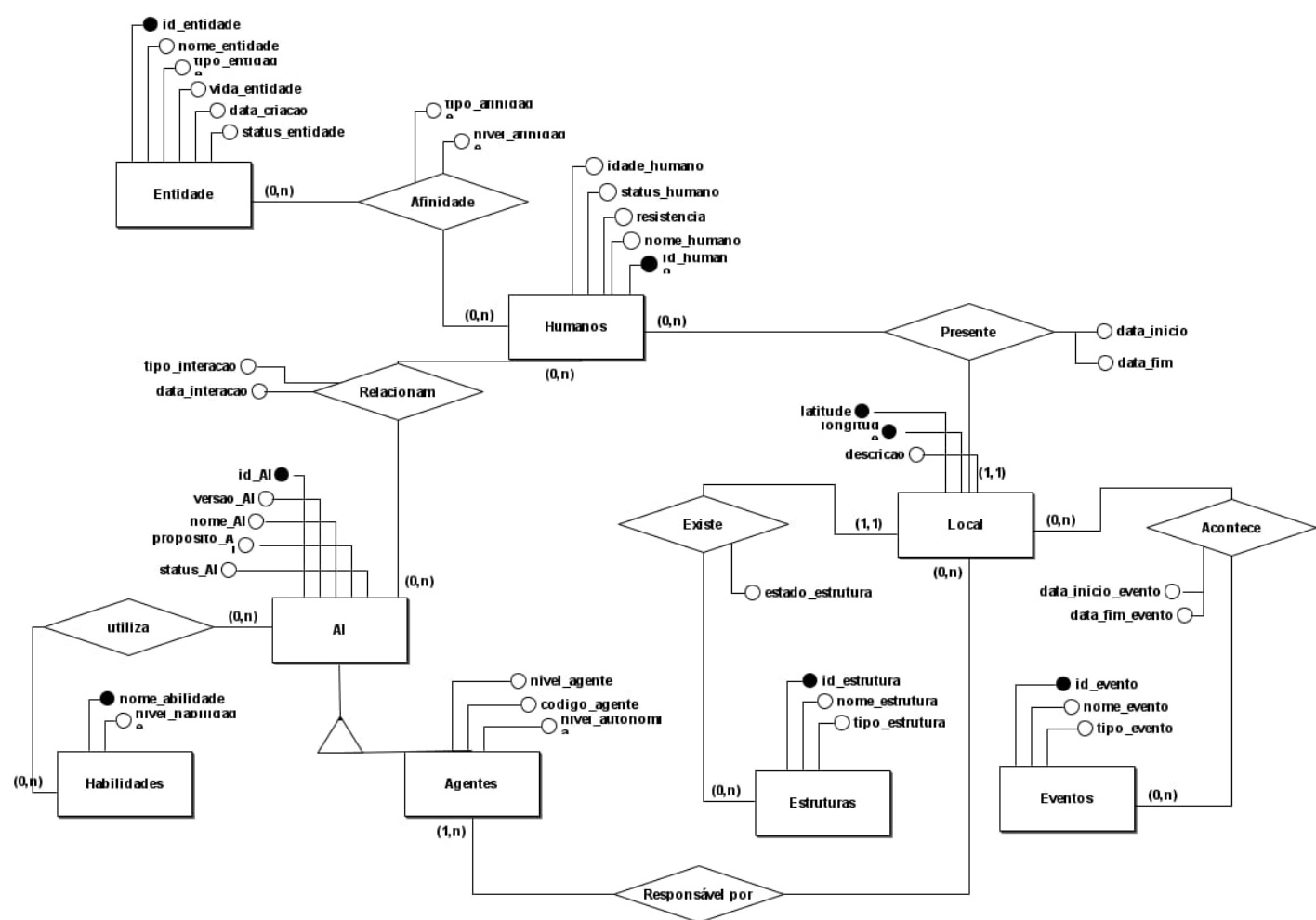


Figura 1: Diagrama ER

Justificativa das Restrições de Integridade Referencial (Chave Estrangeira) Utilizadas:

1. Agentes:

- **fk_id_AI FOREIGN KEY REFERENCES AI (id_AI):** Essa restrição assegura que cada agente esteja associado a uma AI válida. Como os agentes podem ser instâncias de programas ou processos no universo de Matrix, é importante garantir que não existam agentes sem um vínculo adequado a uma AI definida.
- **Ação:** Não há ação de exclusão específica porque é importante que a associação permaneça consistente enquanto a AI existir.

2. Afinidade:

- **fk_id_entidade FOREIGN KEY REFERENCES Entidade (id_entidade) ON DELETE CASCADE:** Isso garante que, ao remover uma entidade, todas as suas relações de afinidade sejam removidas. Esse comportamento é necessário porque uma entidade sem afinidades não faz sentido no contexto.
- **fk_id_humano FOREIGN KEY REFERENCES Humanos (id_humano) ON DELETE CASCADE:** Similarmente, garante que, ao remover um humano, as afinidades relacionadas sejam removidas, mantendo a coerência dos dados.

3. Relacionam:

- `fk_id_humano` **FOREIGN KEY REFERENCES Humanos (id_humano) ON DELETE CASCADE:** Quando um humano é removido, suas interações também são eliminadas, mantendo a nossa lógica, já que um humano inexistente não pode ter interações registradas.
- `fk_id_AI` **FOREIGN KEY REFERENCES AI (id_AI) ON DELETE SET NULL:** Ao remover uma AI, as interações relacionadas permanecem, mas o identificador da AI é definido como `NULL`. Isso preserva o histórico de interações, mesmo que a AI específica não exista mais.

4. Utiliza:

- `fk_id_AI` **FOREIGN KEY REFERENCES AI (id_AI) ON DELETE CASCADE:** Se uma AI for removida, as habilidades associadas a essa AI também devem ser removidas. Esse vínculo garante que não haja habilidades associadas a uma AI inexistente.
- `fk_nome_habilidade` **FOREIGN KEY REFERENCES Habilidades (nome_habilidade) ON DELETE CASCADE:** Se uma habilidade for removida, qualquer associação que uma AI tenha com essa habilidade também é removida, garantindo que não haja associações inválidas.

5. Presente:

- `fk_latitude, fk_longitude` **FOREIGN KEY REFERENCES Lugar (latitude, longitude):** Essa restrição garante que cada presença registrada esteja associada a um local válido. Como as coordenadas latitude e longitude formam uma chave composta, é crucial que todas as presenças estejam mapeadas a locais existentes.
- `fk_id_humano` **FOREIGN KEY REFERENCES Humanos (id_humano) ON DELETE CASCADE:** Ao remover um humano, as presenças associadas também são eliminadas, já que um humano removido não pode estar presente em um lugar.

6. Acontece:

- `fk_latitude, fk_longitude` **FOREIGN KEY REFERENCES Lugar (latitude, longitude) ON DELETE CASCADE:** Se um lugar for removido, todos os eventos que ocorrem nesse lugar também devem ser removidos. Isso é consistente, já que eventos não podem existir sem um local.
- `fk_id_evento` **FOREIGN KEY REFERENCES Eventos (id_evento) ON DELETE CASCADE:** Quando um evento é removido, as relações associadas ao evento também devem ser removidas para evitar registros órfãos.

7. Existe:

- `fk_latitude, fk_longitude` **FOREIGN KEY REFERENCES Lugar (latitude, longitude):** Essa restrição garante que uma estrutura só possa existir em um lugar válido, mantendo a associação geográfica correta.
- `fk_id_estrutura` **FOREIGN KEY REFERENCES Estruturas (id_estrutura) ON DELETE CASCADE:** Ao remover uma estrutura, sua relação com o lugar é removida, garantindo que não haja registros que façam referência a uma estrutura inexistente.

8. ResponsavelPor:

- `fk_latitude, fk_longitude` **FOREIGN KEY REFERENCES Lugar (latitude, longitude) ON DELETE CASCADE:** Se o lugar for removido, a responsabilidade de qualquer agente por esse lugar é eliminada, dando coerência, pois um agente não pode ser responsável por um lugar inexistente.
- `fk_codigo_agente` **FOREIGN KEY REFERENCES Agentes (codigo_agente) ON DELETE CASCADE:** Quando um agente é removido, as responsabilidades que esse agente tem são removidas, garantindo que não restem registros desatualizados.

Script da Criação de Tabelas

Tabelas para as Entidades:

```
CREATE TABLE IF NOT EXISTS Entidade (  
    id_entidade INT PRIMARY KEY,  
    nome_entidade VARCHAR(40) NOT NULL,  
    tipo_entidade VARCHAR(255) NOT NULL,
```

```

        vida_entidade INT DEFAULT 100,
        data_criacao DATE DEFAULT NOW(),
        status_entidade VARCHAR(50) DEFAULT 'Ativo'
    );

CREATE TABLE IF NOT EXISTS Humanos (
    id_humano INT PRIMARY KEY,
    nome_humano VARCHAR(40) NOT NULL,
    -- idade_humano INT,
    data_nascimento TIMESTAMP DEFAULT NOW(),
    status_humano VARCHAR(50) DEFAULT 'online',
    resistencia INT DEFAULT 0 CHECK (resistencia >= 0),
    imagem BYTEA,
        tipo_imagem VARCHAR(10),
        data_upload TIMESTAMP DEFAULT NOW()
    );

CREATE TABLE IF NOT EXISTS Lugar (
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    descricao VARCHAR(255),
    PRIMARY KEY (latitude, longitude)
);

CREATE TABLE IF NOT EXISTS Eventos (
    id_evento INT PRIMARY KEY,
    nome_evento VARCHAR(255) NOT NULL,
    tipo_evento VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS Estruturas (
    id_estrutura INT PRIMARY KEY,
    nome_estrutura VARCHAR(255) NOT NULL,
    tipo_estrutura VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS AI (
    id_AI INT PRIMARY KEY CHECK (id_AI >= 0),
    versao_AI VARCHAR(40) DEFAULT '1.0',
    nome_AI VARCHAR(40) NOT NULL,
    proposito_AI VARCHAR(255) DEFAULT 'Indefinido',
    status_AI VARCHAR(255) DEFAULT 'Ativo'
);

CREATE TABLE IF NOT EXISTS Agentes (
    nivel_agente INT DEFAULT 1 CHECK (nivel_agente >= 0),
    codigo_agente INT UNIQUE,
    nivel_autonomia INT DEFAULT 1,
    fk_id_AI INT NOT NULL,
    CONSTRAINT to_fk_id_AI_Agentes FOREIGN KEY (fk_id_AI) REFERENCES AI (id_AI),
    CONSTRAINT to_pk_codigo_agente_fk_id_AI PRIMARY KEY (codigo_agente, fk_id_AI)
);

CREATE TABLE IF NOT EXISTS Habilidades (
    nome_habilidade VARCHAR(255) PRIMARY KEY,
    nivel_habilidade INT NOT NULL CHECK (nivel_habilidade >= 1)
);

```

Tabelas para os Relacionamentos:


```

CREATE TABLE IF NOT EXISTS Afinidade (
    fk_id_entidade INT REFERENCES Entidade (id_entidade) ON DELETE CASCADE,
    fk_id_humano INT REFERENCES Humanos (id_humano) ON DELETE CASCADE,
    nivel_afinidade INT NOT NULL,
    tipo_afinidade VARCHAR(255) NOT NULL,
    CONSTRAINT to_pk_fk_id_entidade_fk_id_humano
    PRIMARY KEY (fk_id_entidade, fk_id_humano)
);

CREATE TABLE IF NOT EXISTS Relacionam (
    fk_id_humano INT,
    fk_id_AI INT NULL,
    data_interacao DATE DEFAULT NOW(),
    tipo_interacao VARCHAR(255) NOT NULL,
    CONSTRAINT to_fk_id_humano_relacionam FOREIGN KEY (fk_id_humano)
    REFERENCES Humanos (id_humano) ON DELETE CASCADE,
    CONSTRAINT to_fk_id_AI_relacionam FOREIGN KEY (fk_id_AI)
    REFERENCES AI (id_AI) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Utiliza (
    fk_id_AI INT,
    fk_nome_habilidade VARCHAR(255),
    CONSTRAINT to_fk_id_AI_Utiliza FOREIGN KEY (fk_id_AI)
    REFERENCES AI (id_AI) ON DELETE CASCADE,
    CONSTRAINT to_fk_nome_habilidade_Utiliza FOREIGN KEY (fk_nome_habilidade)
    REFERENCES Habilidades (nome_habilidade) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Presente (
    fk_latitude DECIMAL(9,6),
    fk_longitude DECIMAL(9,6),
    fk_id_humano INT,
    data_fim TIMESTAMP,
    data_inicio TIMESTAMP DEFAULT NOW(),
    CONSTRAINT to_fk_lugar_presente FOREIGN KEY (fk_latitude, fk_longitude)
    REFERENCES Lugar (latitude, longitude),
    CONSTRAINT to_fk_humanos_presente FOREIGN KEY (fk_id_humano)
    REFERENCES Humanos (id_humano) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Acontece (
    fk_latitude DECIMAL(9,6),
    fk_longitude DECIMAL(9,6),
    fk_id_evento INT,
    data_inicio_evento DATE DEFAULT NOW(),
    data_fim_evento DATE,
    CONSTRAINT to_fk_lugar_acontece FOREIGN KEY (fk_latitude, fk_longitude)
    REFERENCES Lugar (latitude, longitude) ON DELETE CASCADE,
    CONSTRAINT to_fk_evento_acontece FOREIGN KEY (fk_id_evento)
    REFERENCES Eventos (id_evento) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Existe (
    fk_latitude DECIMAL(9,6),
    fk_longitude DECIMAL(9,6),
    fk_id_estrutura INT REFERENCES Estruturas (id_estrutura)
    ON DELETE CASCADE,
    estado_estrutura VARCHAR(50),

```

```

        CONSTRAINT to_fk_lugar_existe FOREIGN KEY (fk_latitude, fk_longitude)
        REFERENCES Lugar (latitude, longitude)
    );

CREATE TABLE IF NOT EXISTS ResponsavelPor (
    fk_codigo_agente INT,
    fk_latitude DECIMAL(9,6),
    fk_longitude DECIMAL(9,6),
    CONSTRAINT to_fk_lugar_ResponsavelPor
    FOREIGN KEY (fk_latitude, fk_longitude)
    REFERENCES Lugar (latitude, longitude) ON DELETE CASCADE,
    CONSTRAINT to_fk_codigo_agente_ResponsavelPor
    FOREIGN KEY (fk_codigo_agente)
    REFERENCES Agentes (codigo_agente) ON DELETE CASCADE
);

```

Criação das sequências para as entidades:

```

CREATE SEQUENCE IF NOT EXISTS entidade_seq
    AS INT
    INCREMENT BY 2
    START WITH 0
    MINVALUE 0
    NO MAXVALUE
    CACHE 5
    NO CYCLE
    OWNED BY Entidade.id_entidade;
;
ALTER TABLE Entidade
ALTER COLUMN id_entidade SET DEFAULT nextval('entidade_seq');

CREATE SEQUENCE IF NOT EXISTS humanos_seq
    AS INT
    INCREMENT BY 1
    START WITH 0
    MINVALUE 0
    NO MAXVALUE
    CACHE 5
    NO CYCLE
    OWNED BY Humanos.id_humano;
;
ALTER TABLE Humanos
ALTER COLUMN id_humano SET DEFAULT nextval('humanos_seq');

CREATE SEQUENCE IF NOT EXISTS eventos_seq
    AS INT
    INCREMENT BY 10
    START WITH 10
    MINVALUE 0
    NO MAXVALUE
    CACHE 5
    NO CYCLE
    OWNED BY Eventos.id_evento;
;
ALTER TABLE Eventos
ALTER COLUMN id_evento SET DEFAULT nextval('eventos_seq');

CREATE SEQUENCE IF NOT EXISTS estruturas_seq

```

```
AS INT
INCREMENT BY 10
START WITH 10
MINVALUE 0
NO MAXVALUE
CACHE 5
NO CYCLE
OWNED BY Estruturas.id_estrutura;
;
ALTER TABLE Estruturas
ALTER COLUMN id_estrutura SET DEFAULT nextval('estruturas_seq');
```

Carga Inicial:

```
INSERT INTO Entidade (nome_entidade, tipo_entidade, vida_entidade, status_entidade)
VALUES
    ('Cães de Guarda', 'Animais Domésticos', 80, 'Vigilantes'),
    ('Gatos de Rua', 'Animais Selvagens', 90, 'Caçadores'),
    ('Pombos Urbanos', 'Aves', 70, 'Observadores'),
    ('Ratos de Esgoto', 'Roedores', 60, 'Exploradores'),
    ('Morcegos Noturnos', 'Mamíferos Voadores', 85, 'Navegadores');

INSERT INTO Humanos (nome_humano, data_nascimento, status_humano, resistencia)
VALUES
    ('Neo', '1977-09-13', 'Online', 80),
    ('Trinity', '1980-05-27', 'Online', 85),
    ('Morpheus', '1965-03-17', 'Offline', 70),
    ('Tank', '1972-08-12', 'Offline', 60),
    ('Apoc', '1975-06-30', 'Offline', 65);

INSERT INTO Lugar (latitude, longitude, descricao)
VALUES
    (-33.8679, 151.2073, 'Sydney, Austrália'),
    (37.7749, -122.4194, 'San Francisco, EUA'),
    (51.5074, -0.1278, 'Londres, Reino Unido'),
    (48.8566, 2.3522, 'Paris, França'),
    (35.6895, 139.7670, 'Tóquio, Japão');

INSERT INTO Eventos (nome_evento, tipo_evento)
VALUES
    ('Primeira Consciencialização', 'Despertar'),
    ('Encontro com Morpheus', 'Revelação'),
    ('Treinamento na Simulação', 'Preparação'),
    ('Batalha contra Agent Smith', 'Combate'),
    ('Viagem à Cidade das Máquinas', 'Exploração');

INSERT INTO Estruturas (nome_estrutura, tipo_estrutura)
VALUES
    ('Nebuchadnezzar', 'Navio'),
    ('Zion', 'Cidade Subterrânea'),
    ('Templo do Oráculo', 'Edifício'),
    ('Clube Hel', 'Local de Entretenimento'),
    ('Complexo de Produção Humana', 'Fábrica');

INSERT INTO AI (id_AI, versao_AI, nome_AI, proposito_AI, status_AI)
VALUES
    (1, '1.0', 'Oráculo', 'Assessoria', 'Ativo'),
```



```
(2, '2.0', 'Agente Smith', 'Segurança', 'Ativo'),
(3, '1.5', 'Merovíngio', 'Controle de Sistemas', 'Dormindo'),
(4, '2.5', 'Rogue Program', 'Manutenção', 'Ativo'),
(5, '1.2', 'Trainman', 'Controle de Transporte', 'Ativo');

INSERT INTO Agentes (nivel_agente, codigo_agente, nivel_autonomia, fk_id_AI)
VALUES
    (1, 101, 5, 2),
    (2, 102, 5, 2),
    (3, 103, 5, 2),
    (1, 201, 3, 3),
    (2, 301, 4, 4);

INSERT INTO Habilidades (nome_habilidade, nivel_habilidade)
VALUES
    ('Hacking Avançado', 5),
    ('Luta Corporal', 4),
    ('Conhecimento de Sistemas', 3),
    ('Invisibilidade Temporária', 2),
    ('Manipulação de Código', 1),
    ('Hackear o computador da Mirella ', 10);

INSERT INTO Afinidade (fk_id_entidade, fk_id_humano, nivel_afinidade, tipo_afinidade)
VALUES
    (1, 1, 8, 'Empático'),
    (2, 2, 6, 'Intelectual'),
    (3, 3, 7, 'Emocional'),
    (4, 4, 5, 'Prático'),
    (5, 5, 9, 'Espiritual');

INSERT INTO Relacionam (fk_id_humano, fk_id_AI, data_interacao, tipo_interacao)
VALUES
    (1, 1, '2023-01-01', 'Confronto'),
    (2, 2, '2023-02-15', 'Negociação'),
    (3, 3, '2023-03-20', 'Cooperação'),
    (4, 4, '2023-04-10', 'Investigação'),
    (5, 5, '2023-05-25', 'Recrutamento');

INSERT INTO Utiliza (fk_id_AI, fk_nome_habilidade)
VALUES
    (1, 'Hacking Avançado'),
    (2, 'Manipulação de Código'),
    (3, 'Conhecimento de Sistemas'),
    (4, 'Luta Corporal'),
    (5, 'Invisibilidade Temporária');
-- gil isso aqui é do filme 2 caso esteja se perguntando

INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES
    (-33.8679, 151.2073, 1, '2023-01-01 08:30:00'),
    (37.7749, -122.4194, 2, '2023-02-15 09:00:00'),
    (51.5074, -0.1278, 3, '2023-03-20 10:15:00'),
    (48.8566, 2.3522, 4, '2023-04-10 07:00:00'),
    (35.6895, 139.7670, 5, '2023-05-25 11:30:00');

INSERT INTO Acontece
(fk_latitude, fk_longitude, fk_id_evento, data_inicio_evento, data_fim_evento)
```

```
VALUES
    (-33.8679, 151.2073, 1, '2023-01-01', '2023-01-31'),
    (37.7749, -122.4194, 2, '2023-02-15', '2023-03-15'),
    (51.5074, -0.1278, 3, '2023-03-20', '2023-04-20'),
    (48.8566, 2.3522, 4, '2023-04-10', '2023-05-10'),
    (35.6895, 139.7670, 5, '2023-05-25', '2023-06-24');

INSERT INTO
Existe (fk_latitude, fk_longitude, fk_id_estrutura, estado_estrutura)
VALUES
    (-33.8679, 151.2073, 1, 'Operacional'),
    (37.7749, -122.4194, 2, 'Em Construção'),
    (51.5074, -0.1278, 3, 'Desativado'),
    (48.8566, 2.3522, 4, 'Manutenção'),
    (35.6895, 139.7670, 5, 'Seguro');

INSERT INTO ResponsavelPor (fk_codigo_agente, fk_latitude, fk_longitude)
VALUES
    (101, -33.8679, 151.2073),
    (102, 37.7749, -122.4194),
    (103, 51.5074, -0.1278),
    (201, 48.8566, 2.3522),
    (301, 35.6895, 139.7670);

INSERT INTO Humanos (nome_humano, data_nascimento, status_humano, resistencia)
VALUES
    ('Gil', '2004-04-04', 'Online', 44),
    ('Ezio', '2003-03-03', 'Online', 33),
    ('Marcelo', '2002-02-02', 'Online', 55),
    ('Akemi', '2005-05-05', 'Offline', 22);
INSERT INTO Humanos (nome_humano, data_nascimento, status_humano, resistencia)
VALUES
    ('Joao Gabriel', '2006-06-06', 'Online', 1),
    ('Julia Veloso', '2008-03-29', 'Online', 26),
    ('Igao', '2009-09-09', 'Offline', 15),
    ('Bahia', '2010-10-10', 'Offline', 16),
    ('Matheus', '2011-11-11', 'Online', 17);
```

Índices do Banco:

```
CREATE INDEX nomes ON Humanos (nome_humano);
CREATE INDEX codigos ON Agentes (codigo_agente);
```

Justificativa: Em teoria haverá muitos humanos e agentes, e caso as máquinas desejem prosseguir com a proposta do banco, esses índices serão extremamente úteis na consulta dessas entidades.

Visões do Banco:

Interações entre Humanos e Inteligências Artificiais

```
CREATE VIEW View_Humanos_Interacao_AI AS
```

```
SELECT h.nome_humano, h.id_humano,
r.tipo_interacao, r.data_interacao, ai.id_AI, ai.nome_AI
FROM Humanos h JOIN Relacionam r ON h.id_humano = r.fk_id_humano
JOIN AI ai ON r.fk_id_AI = ai.id_AI;
```

Humanos Presentes em Locais e Períodos de Permanência

```
CREATE OR REPLACE VIEW View_Humanos_Presentes AS
SELECT h.nome_humano, h.id_humano, p.fk_latitude AS latitude_local,
p.fk_longitude AS longitude_local, l.descricao AS descricao_local,
p.data_inicio, p.data_fim
FROM Humanos h JOIN Presente p ON h.id_humano = p.fk_id_humano
JOIN Lugar l ON p.fk_latitude = l.latitude AND p.fk_longitude = l.longitude
ORDER BY h.id_humano, p.data_inicio;
```

Procedures no banco:

Procedure para adicionar um novo evento e associá-lo a um local:

```
CREATE OR REPLACE PROCEDURE adicionar_evento_e_local(
    p_nome_evento VARCHAR(255),
    p_tipo_evento VARCHAR(255),
    p_latitude DECIMAL(9,6),
    p_longitude DECIMAL(9,6),
    p_descricao_lugar VARCHAR(255)
)
LANGUAGE plpgsql
AS $$
DECLARE
    lugar_id INT;
    evento_id INT;
BEGIN

    INSERT INTO Lugar (latitude, longitude, descricao)
    VALUES (p_latitude, p_longitude, p_descricao_lugar)
    ON CONFLICT (latitude, longitude) DO UPDATE SET descricao = p_descricao_lugar
    RETURNING id INTO lugar_id;

    INSERT INTO Eventos (nome_evento, tipo_evento)
    VALUES (p_nome_evento, p_tipo_evento)
    RETURNING id_evento INTO evento_id;

    -- Associa o evento ao local
    INSERT INTO Acontece (fk_latitude, fk_longitude, fk_id_evento, data_inicio_evento)
    VALUES (p_latitude, p_longitude, evento_id, CURRENT_DATE);

    RAISE NOTICE 'Evento % adicionado com sucesso.', p_nome_evento;
END;
$;
```

Procedure para atualizar o status de múltiplos agentes:

```
CREATE OR REPLACE PROCEDURE atualizar_status_agentes(
    p_ids_ai INT[],
```

```

        p_novo_status VARCHAR(255)
    )
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE AI
    SET status_AI = p_novo_status
    WHERE id_AI = ANY(p_ids_ai);

    RAISE NOTICE '% agentes atualizados para o status "%".',
    array_length(p_ids_ai, 1), p_novo_status;
END;
$$;

```

Triggers do Banco:

Trigger para logar alterações nos status dos agentes:

```

CREATE TABLE IF NOT EXISTS Log_Status_Agentes (
    id_log INT PRIMARY KEY,
    id_AI INT,
    antigo_status VARCHAR(255),
    novo_status VARCHAR(255),
    data_alteracao TIMESTAMP
);

CREATE SEQUENCE IF NOT EXISTS Log_Status_Agentes_seq
AS INT
INCREMENT BY 1
START WITH 1
MINVALUE 0
NO MAXVALUE
CACHE 5
NO CYCLE
OWNED BY Log_Status_Agentes.id_log;
;

CREATE OR REPLACE FUNCTION logar_alteracao_status_agente()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status_AI <> OLD.status_AI THEN
        INSERT INTO Log_Status_Agentes (id_AI, antigo_status,
        novo_status, data_alteracao)
        VALUES (OLD.id_AI, OLD.status_AI, NEW.status_AI, CURRENT_TIMESTAMP);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_logar_alteracao_status_agente
AFTER UPDATE OF status_AI ON AI
FOR EACH ROW EXECUTE FUNCTION logar_alteracao_status_agente();

```

Trigger para atualizar automaticamente o status de um agente quando seu nível de autonomia aumenta

```

CREATE OR REPLACE FUNCTION atualizar_status_autonomia()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.nivel_autonomia > OLD.nivel_autonomia THEN

```



```

        IF NEW.nivel_autonomia <= 3 THEN
            UPDATE AI
            SET status_AI = 'Supervisionado'
            WHERE id_AI = NEW.fk_id_AI;
        ELSIF NEW.nivel_autonomia BETWEEN 4 AND 6 THEN
            UPDATE AI
            SET status_AI = 'Autônomo'
            WHERE id_AI = NEW.fk_id_AI;
        ELSE
            UPDATE AI
            SET status_AI = 'Perigosamente Autônomo'
            WHERE id_AI = NEW.fk_id_AI;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_atualizar_status_autonomia
AFTER UPDATE OR INSERT ON Agentes
FOR EACH ROW EXECUTE FUNCTION atualizar_status_autonomia();

```

Trigger para atualizar o campo data_fim do registro anterior do humano na tabela Presente sempre que um novo registro for inserido para o mesmo humano

```

CREATE OR REPLACE FUNCTION atualizar_data_fim()
RETURNS TRIGGER AS $$
BEGIN
    -- Atualiza o último registro de presença do humano se data_fim for NULL
    UPDATE Presente
    SET data_fim = NEW.data_inicio
    WHERE fk_id_humano = NEW.fk_id_humano
    AND data_fim IS NULL;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_atualizar_data_fim
BEFORE INSERT ON Presente
FOR EACH ROW
EXECUTE FUNCTION atualizar_data_fim();

```

```

-- Inserts para teste:
-- OBs: Inserir uma por uma
INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES (-33.8679, 151.2073, 1, '2023-01-01 09:30:00');

INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES (37.7749, -122.4194, 2, '2023-02-15 10:00:00');

INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES (51.5074, -0.1278, 3, '2023-03-20 12:15:00');

INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES (48.8566, 2.3522, 4, '2023-04-11 07:00:00');

```

```
INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
VALUES (35.6895, 139.7670, 5, '2023-06-25 11:30:00');
```

Transações no Banco:

Adicionar um novo agente e associá-lo-á a um humano.

```
DO $$
DECLARE
    agente_codigo INT;
    humano_id INT;
BEGIN
    -- Iniciar a transação
    BEGIN
        -- Adicionar novo agente e capturar o código do agente
        INSERT INTO Agentes (nivel_agente, nivel_autonomia, fk_id_AI)
        VALUES (1, 1, 1)
        RETURNING codigo_agente INTO agente_codigo;

        -- Associar o agente a um humano aleatório e capturar o ID do humano
        INSERT INTO Relacionam (fk_id_humano, fk_id_AI, tipo_interacao)
        VALUES (
            (SELECT id_humano FROM Humanos ORDER BY RANDOM() LIMIT 1),
            1,
            'Monitoramento'
        )
        RETURNING fk_id_humano INTO humano_id;

        -- Atualizar o status do agente
        CALL atualizar_status_agentes(ARRAY[agente_codigo], 'Novo Agente');

        -- Commitar a transação
        COMMIT;
    EXCEPTION
        WHEN OTHERS THEN
            -- Em caso de erro, fazer rollback
            ROLLBACK;
            RAISE NOTICE 'Transação falhou: %', SQLERRM;
    END;
END $$;
```

Transação para mover um grupo de humanos para um novo local:

```
BEGIN;
    -- Selecionar humanos aleatórios
    SELECT ARRAY_AGG(id_humano) INTO :humanos_mover
    FROM Humanos
    ORDER BY RANDOM()
    LIMIT 5;

    -- Obter coordenadas aleatórias para o novo local
    SELECT random() * 180 - 90, random() * 360 - 180 INTO :nova_lat, :nova_long;

    -- Adicionar novo local
    INSERT INTO Lugar (latitude, longitude, descricao)
    VALUES (:nova_lat, :nova_long, 'Novo Esconderijo')
    ON CONFLICT (latitude, longitude) DO UPDATE SET descricao = 'Novo Esconderijo';
```

```

-- Mover humanos para o novo local
INSERT INTO Presente (fk_latitude, fk_longitude, fk_id_humano, data_inicio)
SELECT :nova_lat, :nova_long, id_humano, CURRENT_TIMESTAMP
FROM unnest(:humanos_mover) AS id_humano
ON CONFLICT (fk_latitude, fk_longitude, fk_id_humano)
DO UPDATE SET data_inicio = CURRENT_TIMESTAMP;

-- Criar evento de movimentação
PERFORM adicionar_evento_e_local('Movimentação de Grupo',
'Estratégico', :nova_lat, :nova_long, 'Novo Esconderijo');

-- Commitar a transação
COMMIT;
EXCEPTION
WHEN OTHERS THEN
    ROLLBACK;
    RAISE NOTICE 'Transação falhou: %', SQLERRM;
END;

```

TO DO:

1. Instalar NodeJS (Windows):

- Baixar o instalador: <https://nodejs.org/pt>
- Executar o instalador
- Clicar em “Próximo” e aceitar condições.
- Clicar em “Instalar”

2. Baixar Backup do banco

- Baixar Backup anexado
- No PgAdmin criar novo banco de dados nomeado “BD2Matrix”
- Com o botão direito sobre ele selecionar “Restore”
- Buscar pelo backup baixado e confirmar.

2. Baixar código

- Baixar arquivo compactado anexado
- Extrair arquivo
- Abrir pasta e então abrir arquivo “.env”
- Mudar valor da variável “DB_PASSWORD” para a senha do seu usuário postgres.

3. Iniciar Servidor

- Sobre a pasta do projeto, com o botão direito + Shift, selecionar “Abrir com o Powershell”
- Digitar `npm init -y` e apertar enter
- Instalar módulos do necessários com os seguintes comandos:

- `npm i express`
- `npm i pg`
- `npm i dotenv`
- `npm i express-session`
- `npm i body-parser`

- vi. `npm i ejs`
 - vii. `npm i express cors`
 - viii. `npm i pg-format`
 - ix. `npm i argon2`
 - x. `npm i multer`
- d. Iniciar servidor por meio do comando `npm run start`

4. Para acessar a interface, na caixa de pesquisa do seu navegador, digite `localhost:3000/login`

Curiosidade:



Para não ser necessário iniciar o servidor com `npm run start` ou `node server.js` toda vez que alterações forem feitas, utilizaremos o nodemon para ele reinicializar automaticamente

`{}` package.json

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
},
```

`npm i -D nodemon`

Deve ficar assim:

```
PS C:\Users\gilan\Downloads\Nova pasta (32)\bd2> npm i -D nodemon  
npm WARN idealTree Removing dependencies.nodemon in favor of devDependencies.nodemon  
  
removed 3596 packages, changed 1 package, and audited 117 packages in 2m  
  
18 packages are looking for funding  
run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\Users\gilan\Downloads\Nova pasta (32)\bd2>
```

Para iniciar o servidor, digite `npm run dev`